GUARDIA

CYBERSECURITY SCHOOL

CAHIER DES CHARGES

Conception & Reverse Composants

INTRODUCTION

Nom du module : Conception & Reverse Composants
Nom de l'UE : Reverse engineering
Nombre de crédits ECTS : 2
Date : 2 au 13 mars 2026
Classe : GCS2

MODALITÉS

- Groupes de 4 étudiants maximum constitué avant le début du projet
- Aucun livrable ne devra être issu de l'utilisation d'une IA

CONTEXTE ET DESCRIPTION DU PROJET

La rétroconception matérielle de niveau le plus bas consiste à attaquer directement le fonctionnement interne des composants électroniques.

En effet, la rétroconception matérielle cible, dans un premier temps, le système puis, dans un second temps, ses logiciels embarqués. Mais lorsque le logiciel n'est pas accessible ou que des mécanismes de sécurité sont en place, il devient nécessaire de remonter jusqu'à la source, c'est-à-dire l'intérieur même du composant électronique. Il faut alors comprendre l'agencement des éléments internes (portes logiques) et en déduire les fonctions et l'architecture implémentées. Le résultat est ensuite synthétisé dans un langage informatique (débuggable par les mêmes outils que le système physique ; VHDL) permettant de simuler le comportement du système étudié.

Le projet a pour objectif de familiariser les étudiants avec la conception matérielle à travers la conception d'un système de chiffrement dérivé d'un algorithme largement utilisé.

Guardia CS - © Active Learning by Quest Education Group
Document Interne 2025-2026

L’utilisation du logiciel FALSTAD permettra aux étudiants de comprendre le rôle de chaque composant « de base » et celui de combinaisons de composants de base fréquemment implémentées dans les composants réels.

Un CTF hardware constituera le second volet de ce projet : une dizaine d’énigmes de difficulté croissante devra être résolue. Il ne s’agira pas uniquement de « trouver le flag » mais davantage d’identifier et de s’approprier les méthodes de rétroconception de composants.

TRAVAUX PRÉPARATOIRES

Vous serez **évalués en début de module** à travers un **questionnaire** sur la base du contenu de cette section. Cette note sera intégrée à votre note de contrôle continu sur chaque projet (voir la rubrique « **ÉVALUATION** » en fin de document).

- Connaître le cours **"UE6-1_Approfondissement au reverse engineering"** (pages 70 à 100)
- Connaître la fiche sur la prise en main de FALSTAD

RESSOURCES

L'examen des ressources citées ci-après est **indispensable** à la bonne réalisation des travaux préparatoires décrits ci-dessus.

Les outils de recherche et logiciels/moteurs d'IA générative existants vous permettront d'**accéder facilement et rapidement** aux essentiels de celles-ci (sans avoir à étudier l'intégralité des documents originels) sur lesquels vous serez évalué en début de module par le biais d'un questionnaire.

Ce travail de recherche via ces outils d'IA constitue d'ailleurs une compétence que vous allez devoir acquérir (ou parfaire) dans le cadre de votre formation.

- Cours « UE6-1_Approfondissement au reverse engineering » (pages 70 à 100)
- Fiche sur la prise en main de FALSTAD

OBJECTIFS PÉDAGOGIQUES

Savoirs / connaissances

- Fonctionnement des composants électroniques
- Implémentation des fonctions logiques basiques
- Rétroconception de composants électroniques

Savoir-faire / Compétences

- Identifier les différentes fonctions implémentées dans un composant

Savoir-être

- Implication
- Persévérance

INTERVENANTE

Nom : Anouchka HAVLIK

Email : anouchka.havlik@gmail.com

BIOGRAPHIE

Expert Forensic numérique spécialisée en réparation de supports électroniques (Gendarmerie Nationale)

ROADMAP

Jalon	Livrables attendus	Date limite	Moyens / formats
1	Quiz initial	02/03/2026	QCM
2	Projet de conception	06/03/2026	Conception d'un système (design FALSTAD, VHDL, dossier explicatif des choix techniques réalisés pour les 2 parties)
3	CTF	13/03/2026	CTF hardware (résolution des différentes énigmes de façon séquentielle ; dossier explicatif pour la résolution de chaque étape)
4	Partiel		

Guardia CS - © Active Learning by Quest Education Group
Document Interne 2025-2026

EVALUATION

Libellé	Pondération (sur 300 points)
Travaux préparatoires	**35**
Questionnaire	35
Posture professionnelle	**15**
Assiduité	2.5
Ponctualité	2.5
Implication	10
Projet CONCEPTION	**30**
Conception FALSTAD	10
Conception VHDL	10
Rapport global de projet	10
CTF hardware	**20**
Rapport du CTF hardware	20
Examen	**200**

TRAVAUX ATTENDUS

PROJET DE CONCEPTION

L'objectif de ce projet est de concevoir un système électronique à l'aide du logiciel FALSTAD puis de le simuler en langage VHDL.
A l'issue de ce projet, vous aurez réalisé une fonction cryptographique courante (légèrement simplifiée à des fins pédagogiques).

Dans un premier temps, vous vous familiariserez avec les « briques » électroniques et fonctions électroniques de base en les concevant, une à une et indépendamment les unes des autres.
Dans un second temps, vous associerez différentes fonctions conçues précédemment afin de créer des fonctions plus complexes.
Vous combinerez par la suite toutes vos fonctions afin d'obtenir un montage cohérent et répondant aux exigences du projet.
Enfin (partie bonus), vous mettrez en évidence les défauts du système conçu et proposerez des solutions pour les corriger.

Le système global devra être conçu pour fonctionner sur un message d'entrée de 16 bits traités par séries de 4 bits à la fois.
Les fonctions conçues devront pouvoir traiter 4 bits à la fois, sauf mention contraire.
<u>Dans un même fichier projet</u>, mais indépendamment les unes des autres, vous réaliserez les différentes fonctions décrites.

- **PARTIE 1**

Vous importerez les éléments de base existants (presque tous) dans FALSTAD et les créerez en VHDL, sous forme de composants indépendants qui pourront être appelés par la suite.

FALSTAD	VHDL
Importez un XOR avec 2 entrées logiques.	
Importez une bascule D avec 1 entrée logique.	Créez un composant **Dgate** et simulez-le.
Importez un multiplexeur avec 3 entrées logiques.	Créez un composant **MUX** avec les mêmes caractéristiques et simulez-le.

- **PARTIE 2**

Vous associerez différents éléments de base importés en Partie 1 afin de créer des fonctions plus élaborées. Il est également possible d'importer d'autres éléments ou d'en créer.

Fonction à créer	**FALSTAD**	**VHDL**
Registre à décalage	Créez un registre qui prend en entrée une valeur et la décale vers la droite à chaque nouveau front d'horloge (fréquence 100 Hz). La fonction créée doit fonctionner sur 4 bits.	Créez un composant **R_right** avec ces caractéristiques et le simuler.
Registre de stockage	Créez un registre qui prend en entrée 4 bits simultanément et les transfère en sortie à chaque front d'horloge (fréquence 25 Hz).	Créez un composant **Buffer** avec ces caractéristiques et le simuler.
Registre de sélection	Créez un registre qui permet simultanément de sélectionner un set de 4 bits d'entrée (parmi 2) et d'acheminer ce set choisi en sortie.	Créez un composant **R_sel** avec ces caractéristiques et le simuler.
Compteur	Créez un compteur permettant de compter 16 valeurs en binaire et dépendant d'une unique horloge cadencée à 25 Hz.	Créez un composant **CPT** avec ces caractéristiques et le simuler.

- **PARTIE 3**

Vous combinerez les différents éléments conçus en Partie 2 afin de créer des fonctions mathématiques. Il s'agit à nouveau d'associer des composants préalablement importés pour créer des fonctions mais il est également possible de créer des éléments.

<u>Dans cette Partie 3, seul le schéma FALSTAD sera attendu.</u>

<table>
<tr><th>Fonction à créer</th><th>FALSTAD</th></tr>
<tr><td>Mix Key</td><td>Créez un registre qui fonctionne avec 4 bits de message simultanément en entrée et qui associe à chaque bit du message 1 bit aléatoire (clé fixe de votre choix) de la manière suivante :
- Bit à bit, si le bit de message et le bit de clé sont identiques, la sortie sera ‘0’ ;
- Bit à bit, si le bit de message et le bit de clé sont différents, la sortie sera ‘1’.</td></tr>
<tr><td>Row Shift</td><td>Créez un registre prenant en entrée 4 bits de message simultanément et les acheminant en sortie de fonction sur 2 fronts d’horloge (fréquence à 25 Hz) sachant qu’entre l’entrée et la sortie de la fonction, chaque ligne de bits devra avoir été décalée d’une position de la manière suivante :

par exemple :
<table>
<tr><td></td><td>Entrée</td><td>→</td><td>Sortie</td><td></td></tr>
<tr><td>b0</td><td>0</td><td></td><td>0</td><td>b0</td></tr>
<tr><td>b1</td><td>1</td><td></td><td>0</td><td>b1</td></tr>
<tr><td>b2</td><td>0</td><td></td><td>1</td><td>b2</td></tr>
<tr><td>b3</td><td>0</td><td></td><td>0</td><td>b3</td></tr>
</table>
ou encore :
<table>
<tr><td></td><td>Entrée</td><td>→</td><td>Sortie</td><td></td></tr>
<tr><td>b0</td><td>0</td><td></td><td>1</td><td>b0</td></tr>
<tr><td>b1</td><td>0</td><td></td><td>0</td><td>b1</td></tr>
<tr><td>b2</td><td>1</td><td></td><td>0</td><td>b2</td></tr>
<tr><td>b3</td><td>1</td><td></td><td>1</td><td>b3</td></tr>
</table></td></tr>
<tr><td>Flip Bit</td><td>Créez un registre prenant en entrée 4 bits de message simultanément et les acheminant en sortie de fonction sur 2 fronts d’horloge (fréquence à 25 Hz) sachant qu’entre l’entrée et la sortie de la fonction, seules 2 lignes de bits devront avoir été échangées entre elles (les 2 autres ne devront subir aucune variation de valeur entre l’entrée et la sortie de la fonction).</td></tr>
<tr><td>Sub Bit</td><td>Créez un registre prenant en entrée 4 bits de message simultanément et acheminant en sortie de fonction sur 2 fronts d’horloge (fréquence à 25 Hz) :
- 3 bits de message sans aucune variation de leur valeur ;
- 1 bit de message avec 1 front d’horloge de retard par rapport aux 3 autres bits et sa valeur inversée.</td></tr>
<tr><td>Load Data</td><td>Créez un registre qui permet de charger un fichier d’entrée avec des données en série puis de fournir en sortie des séries de blocs de 4 bits en parallèle.</td></tr>
</table>

<u>A l’issue de la Partie 3, vous devriez disposer d’un fichier FALSTAD avec 5 blocs indépendants qui réalisent chacun une fonction.</u>

- **PARTIE 4**

Vous assemblerez dans cette partie les 5 blocs obtenus à l'issue de la Partie 3. Il sera nécessaire d'ajouter 2 autres blocs/fonctions pour obtenir l'algorithme final.

Cette Partie 4 s'articulera en 4 phases :

1) Combiner les différents blocs dans le bon ordre ;
2) Insérer un rebouclage nécessaire au bon fonctionnement de l'algorithme
3) Analyser le comportement du système afin d'en ajuster le fonctionnement ;

4) Transposer le schéma FALSTAD en VHDL.

1) Boucle principale :

La boucle principale de l'algorithme devra être séquencée comme suit :

Load Data → Sub Bit → Row Shift → Flip Bit → Mix Key

L'entrée de la boucle principale devra désormais être un fichier contenant un message aléatoire d'une longueur de 16 bits.

La première ligne du fichier devra être la suivante :

time step = 0.000005 sec

Puis le fichier devra contenir 1 bit par ligne.

Les horloges identiques pourront être reliées entre elles.

2) Rebouclage

Afin de complexifier l'algorithme, il serait souhaitable que le calcul réalisé dans la boucle principale soit répété indéfiniment.

Il sera nécessaire d'insérer entre les modules Load Data et Sub Bit, un module permettant de choisir

- soit d'injecter 4 bits du plaintext d'entrée (chargement du plaintext),
- soit d'injecter une sortie du Mix Key (cycle précédent ; rebouclage).

Pour ce faire, vous pourrez utiliser 2 éléments créés dans la Partie 1.

3) Optimisation

Dans notre système, l'un des modules créés permet de charger un plaintext de 16 bits (4 fois 4 bits) qui subit ensuite 4 opérations mathématiques avant d'être rebouclé à l'infini.

On constate un problème dans le rebouclage : compte tenu du trop grand nombre d'étapes dans l'algorithme, l'injection du plaintext se termine « trop tôt ». Il sera nécessaire, sans modifier l'ordre des opérations mathématiques, de faire en sorte que la première série de 4 bits entre dans la phase de rebouclage juste après la fin de l'injection du plaintext.

4) VHDL

Le schéma sous FALSTAD du système étant finalisé et simplifié, vous pourrez le transposer en VHDL.
Pour ce faire, vous appellerez les composants créés dans le bon ordre afin de construire l'algorithme de chiffrement.

Il ne sera pas demandé de le simuler en version VHDL.

BONUS

Afin de maximiser votre note, vous pourrez répondre à l'une des questions bonus ci-après :

a) Créez une sortie du chiffré uniquement après 10 rounds
b) Modifiez la sortie pour avoir 4 séries de 4 bits uniquement à l'issue des 10 rounds

Livrable du projet de conception :

Le livrable est attendu sous forme d'un rapport comportant pour chaque élément présenté, son nom, le schéma ainsi que le code VHDL correspondants et la courbe simulée.
Une attention particulière sera accordée à la qualité et à la présentation du rapport (clarté et intelligibilité des explications et du suivi des schémas, figures et courbes (titres, numérotations, ...)).

Rappelons que bien que les projets soient à réaliser en groupe, les groupes sont indépendants les uns des autres. Par conséquent, aucune mise en commun, entre groupes différents, de schémas FALSTAD, code VHDL, etc., ne sera tolérée.

CTF

Le jour du début du projet, 10 fichiers zip protégés chacun par un mot de passe seront mis à votre disposition.

Le mot de passe du 1er sera donné à tous les groupes au début du challenge.
Vous devrez ensuite résoudre chaque énigme pour trouver le mot de passe permettant d'accéder au fichier suivant. Les mots de passe sont simples, l'évaluation portant prioritairement sur la réflexion mise en œuvre pour la résolution de chaque montage. Aussi, si vous êtes sûrs de votre réponse, n'hésitez pas à vous tourner vers votre intervenant pour vérifier la syntaxe de votre mot de passe afin d'éviter de rester bloqués…

Livrable du CTF :
Le livrable est attendu sous forme d'un rapport dans lequel vous expliquerez la méthode de résolution de chaque énigme.
Pour chaque énigme, vous fournirez le schéma initial, le mot de passe ainsi que des explications détaillées de la méthode de résolution (étape par étape) avec des schémas intermédiaires ainsi qu'une simplification du montage : le montage final attendu devra être le plus simplifié possible.

La clarté des explications de la méthode de résolution ainsi que les simplifications des montages primeront sur le nombre d'énigmes résolues.

Rappelons que bien que les projets soient à réaliser en groupe, les groupes sont indépendants les uns des autres. Par conséquent, aucune mise en commun, entre groupes différents, de schémas FALSTAD, code VHDL, etc., ne sera tolérée.